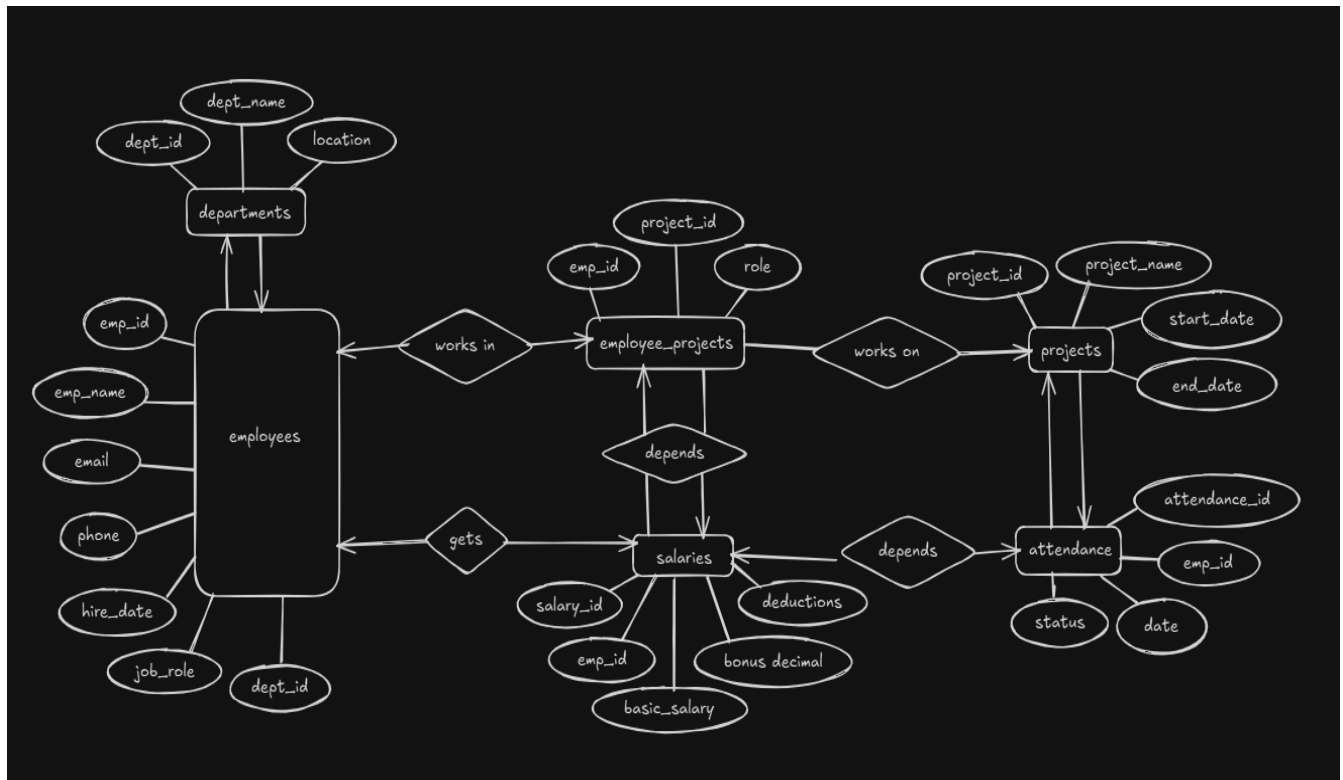


HR & Payroll **Management System**

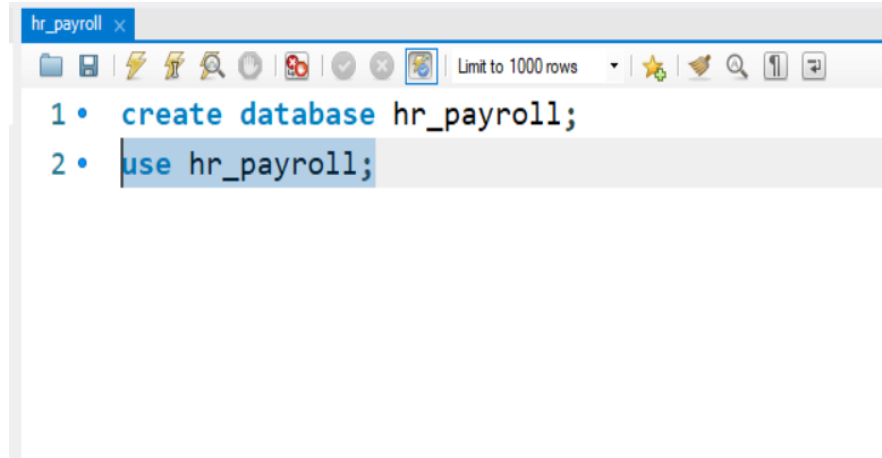
1. ER Diagram (Text Representation)



2. Database Schema (DDL)

2.1 Create Database

```
create database hr_payroll;  
use hr_payroll;
```



2.2 Create Tables

➤ departments

```
create table departments (  
    dept_id int primary key,  
    dept_name varchar(50) unique not null,  
    location varchar(50));
```

➤ employees

```
create table employees (  
    emp_id int primary key,  
    emp_name varchar(100) not null,  
    email varchar(100) unique,  
    phone varchar(15),  
    hire_date date not null,  
    dept_id int,
```

```
job_role varchar(50),  
constraint fk_dept foreign key (dept_id) references  
departments(dept_id),  
constraint chk_role check (job_role in  
('Developer','Manager','HR','Accountant','Tester')));
```

➤ **projects**

```
create table projects (  
project_id int primary key,  
project_name varchar(100) not null,  
start_date date,  
end_date date);
```

➤ **employee_projects (Many-to-Many)**

```
create table employee_projects (  
emp_id int,  
project_id int,  
role varchar(50),  
primary key(emp_id, project_id),  
foreign key(emp_id) references employees(emp_id),  
foreign key (project_id) references projects(project_id));
```

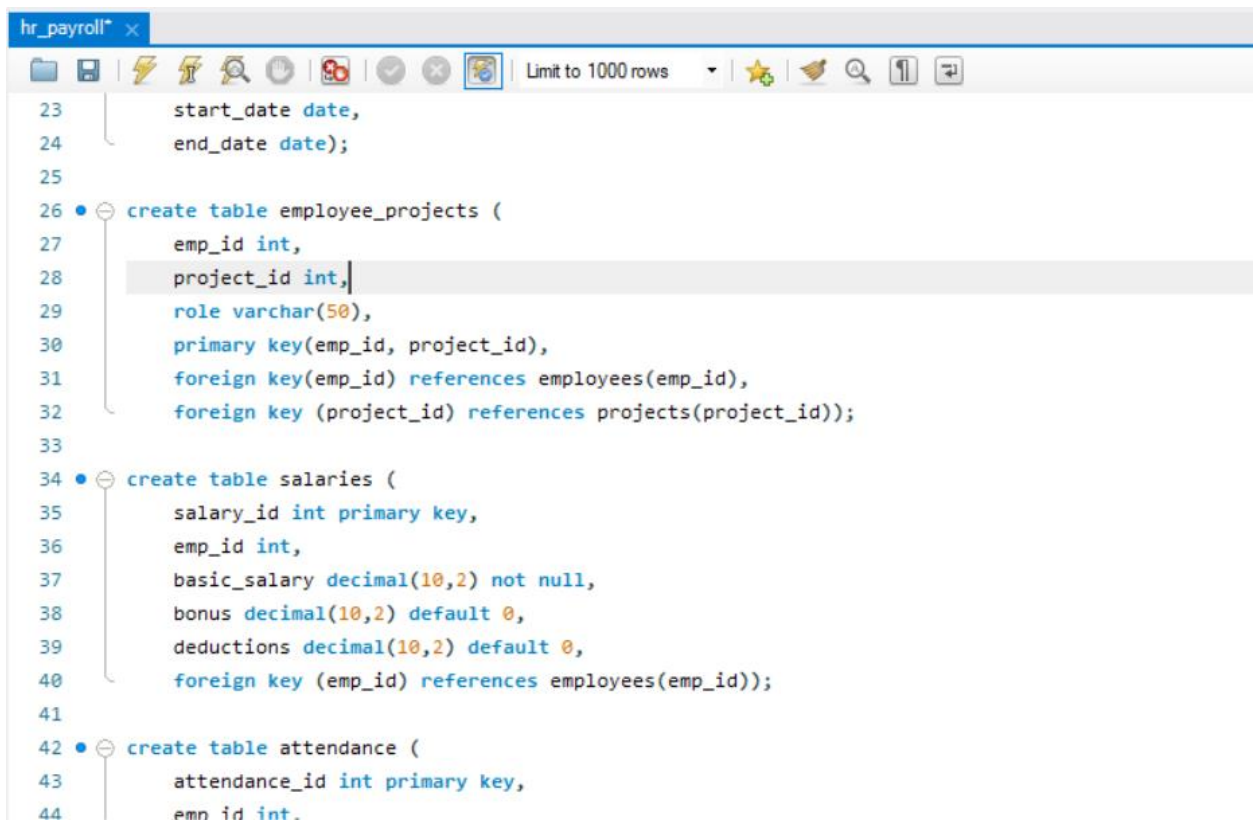
➤ **salaries**

```
create table salaries (  
salary_id int primary key,  
emp_id int,  
basic_salary decimal(10,2) not null,  
bonus decimal(10,2) default 0,  
deductions decimal(10,2) default 0,  
foreign key (emp_id) references employees(emp_id));
```

➤ **attendance**

```
create table attendance (  
attendance_id int primary key,  
emp_id int,  
date date,
```

status varchar(10) check (status in
('Present','Absent','Leave')),
foreign key (emp_id) references employees(emp_id));



```
23     start_date date,  
24     end_date date);  
25  
26 • create table employee_projects (  
27     emp_id int,  
28     project_id int,  
29     role varchar(50),  
30     primary key(emp_id, project_id),  
31     foreign key(emp_id) references employees(emp_id),  
32     foreign key (project_id) references projects(project_id));  
33  
34 • create table salaries (  
35     salary_id int primary key,  
36     emp_id int,  
37     basic_salary decimal(10,2) not null,  
38     bonus decimal(10,2) default 0,  
39     deductions decimal(10,2) default 0,  
40     foreign key (emp_id) references employees(emp_id));  
41  
42 • create table attendance (  
43     attendance_id int primary key,  
44     emp_id int.
```

3. Sample Dataset (DML)

➤ departments

```
insert into departments values  
(1, 'HR', 'Pune'),  
(2, 'Development', 'Pune'),  
(3, 'Finance', 'Mumbai'),  
(4, 'QA', 'Pune'),  
(5, 'Management', 'Mumbai');
```

➤ employees

```
insert into employees values  
(101,'Amit  
Sharma','amit@company.com','9876543210','2022-01-  
10',2,'Developer'),  
(102,'Priya  
Singh','priya@company.com','9922334455','2021-03-  
15',1,'HR'),  
(103,'Rahul  
Verma','rahul@company.com','8811223344','2020-07-  
22',2,'Manager'),  
(104,'Neha  
Joshi','neha@company.com','9988776655','2023-02-  
10',4,'Tester'),  
(105,'Karan  
Patel','karan@company.com','9090909090','2019-11-  
01',3,'Accountant'),  
(106,'Sonia  
Mehta','sonia@company.com','8899001122','2021-04-  
19',2,'Developer');
```

➤ **projects**

```
insert into projects values
(1,'Payroll System','2023-01-01','2023-06-30'),
(2,'E-Commerce Portal','2023-02-01','2023-09-30'),
(3,'Mobile App','2023-03-15','2023-12-31');
```

➤ **employee_projects**

```
insert into employee_projects values
(101,1,'Developer'),
(101,2,'Developer'),
(102,1,'HR Support'),
(103,2,'Manager'),
(106,3,'Developer'),
(104,2,'Tester');
```

➤ **salaries**

```
insert into salaries values
(1,101,60000,5000,2000),
(2,102,45000,3000,1000),
(3,103,80000,7000,5000),
(4,104,40000,2000,500),
(5,105,55000,4000,1500),
(6,106,62000,6000,2500);
```

➤ **attendance**

```
insert into attendance values
(1,101,'2023-12-01','Present'),
(2,102,'2023-12-01','Absent'),
(3,103,'2023-12-01','Present'),
(4,104,'2023-12-01','Leave'),
(5,105,'2023-12-01','Present'),
(6,106,'2023-12-01','Present');
```

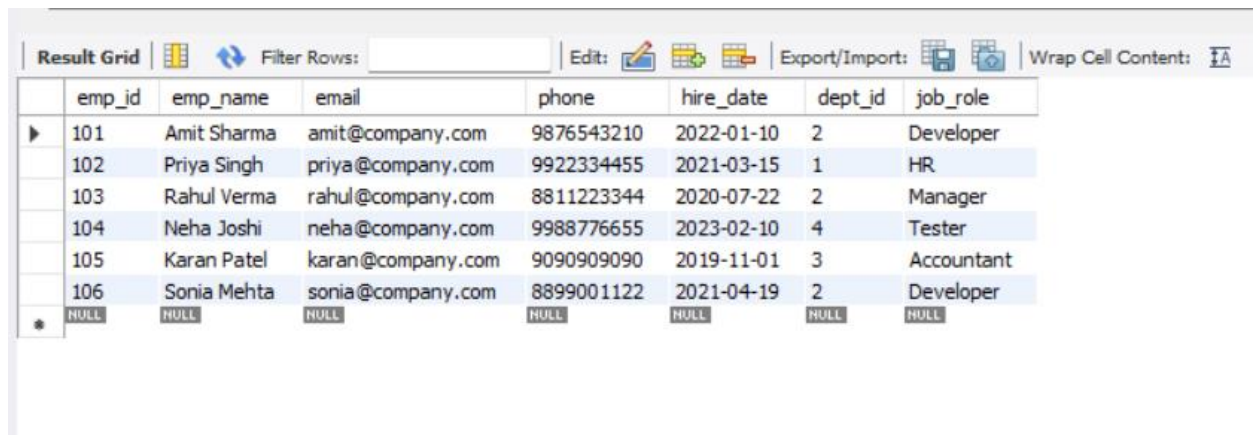
```
hr_payroll* x
Limit to 1000 rows
68
69 • insert into employee_projects values
70 (101,1,'Developer'),
71 (101,2,'Developer'),
72 (102,1,'HR Support'),
73 (103,2,'Manager'),
74 (106,3,'Developer'),
75 (104,2,'Tester');
76
77 • insert into salaries values
78 (1,101,60000,5000,2000),
79 (2,102,45000,3000,1000),
80 (3,103,80000,7000,5000),
81 (4,104,40000,2000,500),
82 (5,105,55000,4000,1500),
83 (6,106,62000,6000,2500);
84
85 • insert into attendance values
86 (1,101,'2023-12-01','Present'),
87 (2,102,'2023-12-01','Absent'),
88 (3,103,'2023-12-01','Present'),
89 (4,104,'2023-12-01','Leave');
```


4. 40+ SQL Tasks - Actual Project

A. Basic Queries

1. List all employees

```
select * from employees;
```

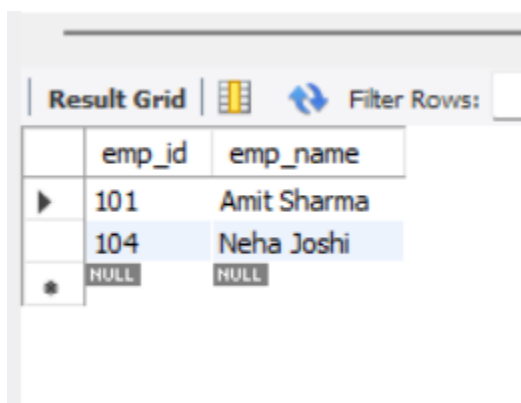


The screenshot shows a SQL query result grid with the following columns: emp_id, emp_name, email, phone, hire_date, dept_id, and job_role. The data is as follows:

	emp_id	emp_name	email	phone	hire_date	dept_id	job_role
▶	101	Amit Sharma	amit@company.com	9876543210	2022-01-10	2	Developer
	102	Priya Singh	priya@company.com	9922334455	2021-03-15	1	HR
	103	Rahul Verma	rahul@company.com	8811223344	2020-07-22	2	Manager
	104	Neha Joshi	neha@company.com	9988776655	2023-02-10	4	Tester
	105	Karan Patel	karan@company.com	9090909090	2019-11-01	3	Accountant
	106	Sonia Mehta	sonia@company.com	8899001122	2021-04-19	2	Developer
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. Employees hired after 2021

```
select emp_id, emp_name from employees where hire_date > "2021-12-31";
```

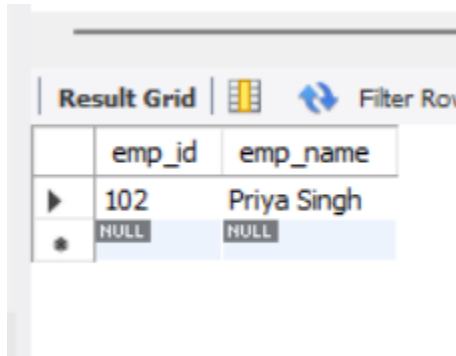


The screenshot shows a SQL query result grid with the following columns: emp_id and emp_name. The data is as follows:

	emp_id	emp_name
▶	101	Amit Sharma
	104	Neha Joshi
*	NULL	NULL

3. HR department employees

```
select emp_id, emp_name from employees where job_role = "HR";
```



	emp_id	emp_name
▶	102	Priya Singh
•	NULL	NULL

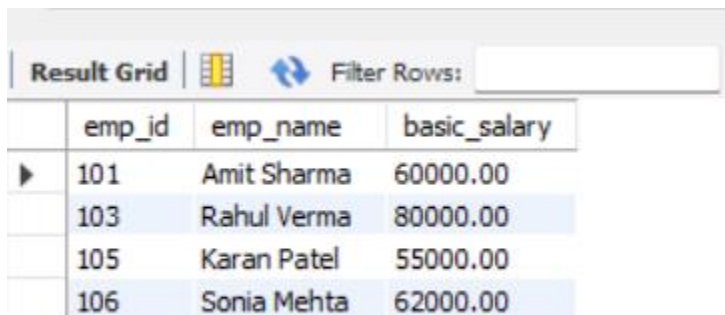
B. Filtering & Sorting

1. Employees with salary > 50,000

```
select employees.emp_id, employees.emp_name, salaries.basic_salary from  
employees
```

```
join salaries on employees.emp_id = salaries.emp_id
```

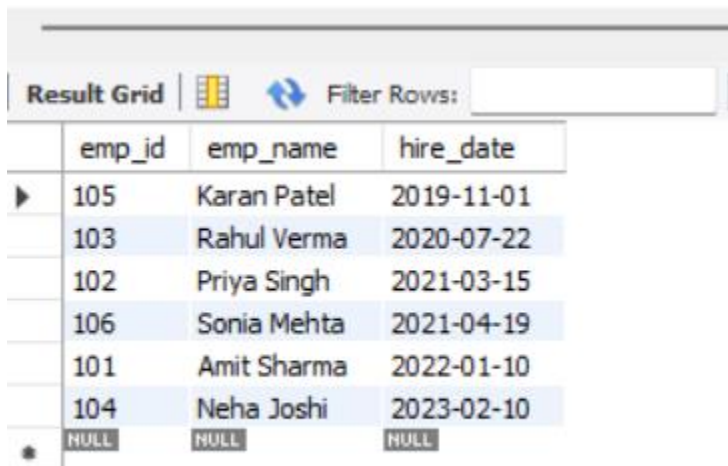
```
where salaries.basic_salary > 50000;
```



	emp_id	emp_name	basic_salary
▶	101	Amit Sharma	60000.00
	103	Rahul Verma	80000.00
	105	Karan Patel	55000.00
	106	Sonia Mehta	62000.00

2. Order employees by hire_date

select emp_id, emp_name, hire_date from employees order by hire_date;

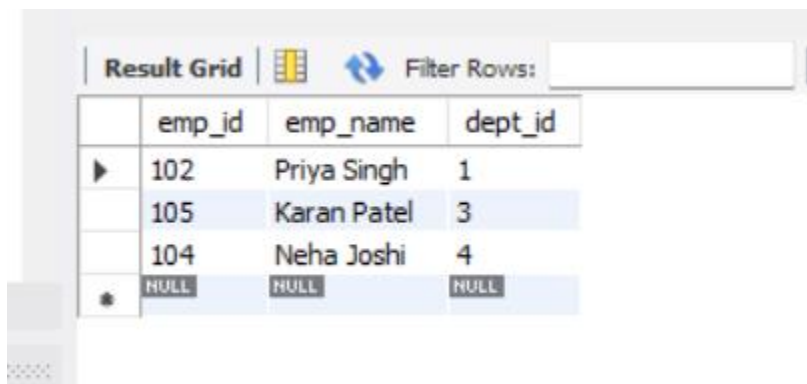


The screenshot shows a 'Result Grid' interface with a 'Filter Rows' input field. The table displays the following data:

	emp_id	emp_name	hire_date
▶	105	Karan Patel	2019-11-01
	103	Rahul Verma	2020-07-22
	102	Priya Singh	2021-03-15
	106	Sonia Mehta	2021-04-19
	101	Amit Sharma	2022-01-10
	104	Neha Joshi	2023-02-10
★	NULL	NULL	NULL

3. Employees not in Development department

select emp_id, emp_name, dept_id from employees where dept_id != 2;



The screenshot shows a 'Result Grid' interface with a 'Filter Rows' input field. The table displays the following data:

	emp_id	emp_name	dept_id
▶	102	Priya Singh	1
	105	Karan Patel	3
	104	Neha Joshi	4
★	NULL	NULL	NULL

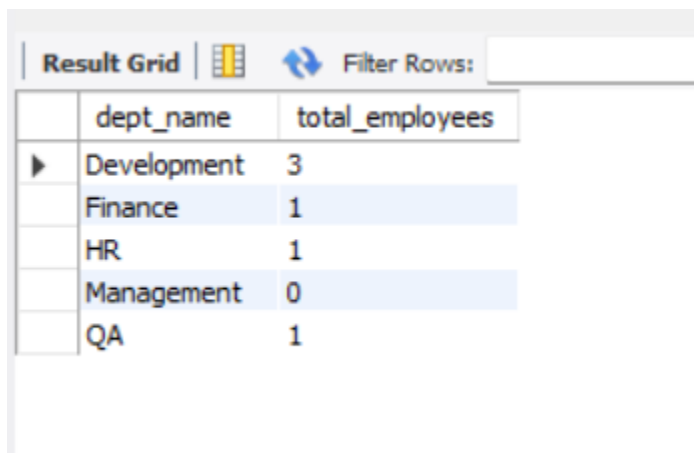
C. Aggregate Functions

1. Count employees in each department

```
select departments.dept_name, count(employees.emp_id) as total_employees  
from departments
```

```
left join employees on departments.dept_id = employees.dept_id
```

```
group by departments.dept_name;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains two columns: 'dept_name' and 'total_employees'. The data is as follows:

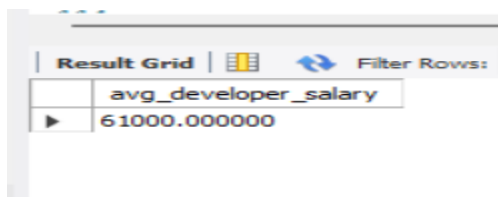
dept_name	total_employees
Development	3
Finance	1
HR	1
Management	0
QA	1

2. Avg salary of developers

```
select avg(salaries.basic_salary) as avg_developer_salary from employees
```

```
join salaries on employees.emp_id = salaries.emp_id
```

```
where employees.job_role = 'Developer';
```

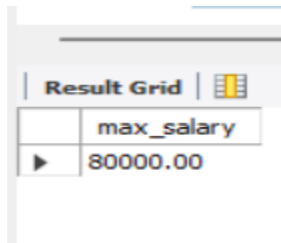


The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains one column: 'avg_developer_salary'. The data is as follows:

avg_developer_salary
61000.000000

3. Max salary in company

```
select max(basic_salary) as max_salary from salaries;
```



A screenshot of a SQL query result grid. The grid has two columns: 'max_salary' and a value '80000.00'. The title bar of the window says 'Result Grid'.

	max_salary
▶	80000.00

D. GROUP BY + HAVING

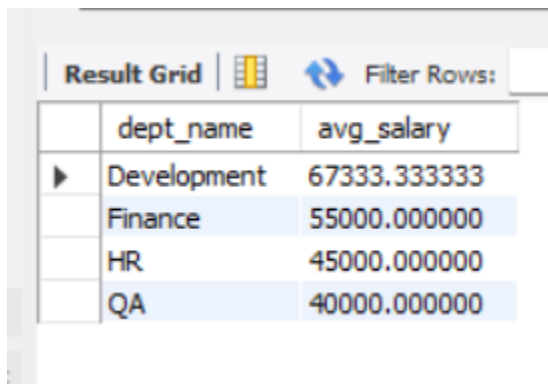
1. Avg salary per department

```
select departments.dept_name, avg(salaries.basic_salary) as avg_salary from  
departments
```

```
join employees on departments.dept_id = employees.dept_id
```

```
join salaries on employees.emp_id = salaries.emp_id
```

```
group by departments.dept_name;
```



A screenshot of a SQL query result grid. The grid has two columns: 'dept_name' and 'avg_salary'. The rows are: Development (67333.333333), Finance (55000.000000), HR (45000.000000), and QA (40000.000000). The title bar of the window says 'Result Grid' and 'Filter Rows:'.

	dept_name	avg_salary
▶	Development	67333.333333
	Finance	55000.000000
	HR	45000.000000
	QA	40000.000000

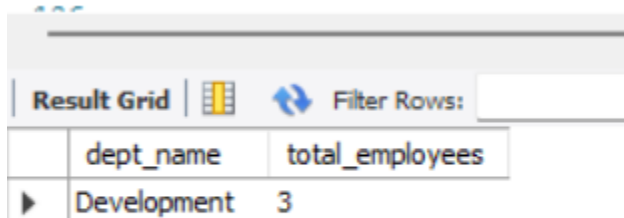
2. Departments with more than 2 employees

```
select departments.dept_name, count(employees.emp_id) as total_employees  
from departments
```

```
join employees on departments.dept_id = employees.dept_id
```

```
group by departments.dept_name
```

```
having count(employees.emp_id) > 2;
```



	dept_name	total_employees
▶	Development	3

E. Joins (All Types)

1. Employees with departments

```
select employees.emp_id, employees.emp_name, departments.dept_name  
from employees
```

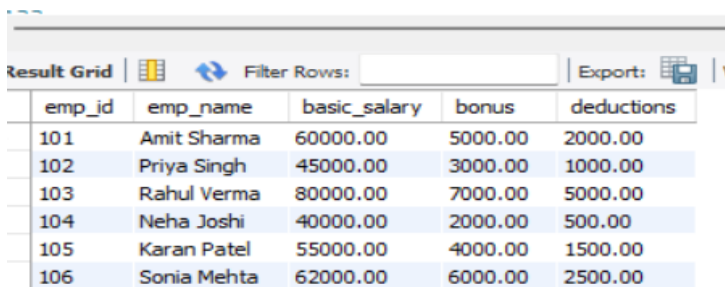
```
join departments on employees.dept_id = departments.dept_id;
```



	emp_id	emp_name	dept_name
▶	101	Amit Sharma	Development
	103	Rahul Verma	Development
	106	Sonia Mehta	Development
	105	Karan Patel	Finance
	102	Priya Singh	HR
	104	Neha Joshi	QA

2. Employees with their salary details

```
select employees.emp_id, employees.emp_name,  
salaries.basic_salary, salaries.bonus, salaries.deductions  
from employees join salaries on employees.emp_id = salaries.emp_id;
```

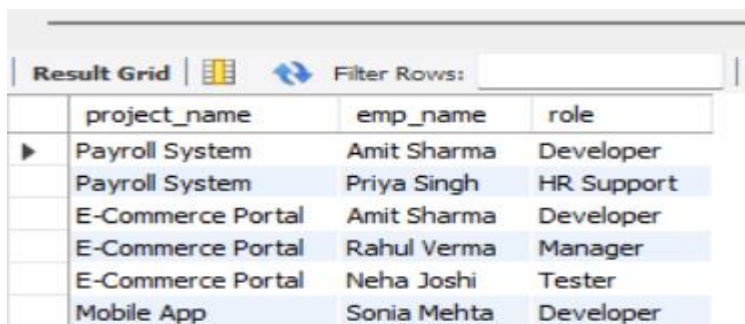


The screenshot shows a 'Result Grid' window with a table containing 6 rows of employee salary data. The table has columns for emp_id, emp_name, basic_salary, bonus, and deductions. The data is as follows:

emp_id	emp_name	basic_salary	bonus	deductions
101	Amit Sharma	60000.00	5000.00	2000.00
102	Priya Singh	45000.00	3000.00	1000.00
103	Rahul Verma	80000.00	7000.00	5000.00
104	Neha Joshi	40000.00	2000.00	500.00
105	Karan Patel	55000.00	4000.00	1500.00
106	Sonia Mehta	62000.00	6000.00	2500.00

3. Projects with assigned employees

```
select projects.project_name, employees.emp_name, employee_projects.role  
from projects join employee_projects  
on projects.project_id = employee_projects.project_id  
join employees on employee_projects.emp_id = employees.emp_id;
```

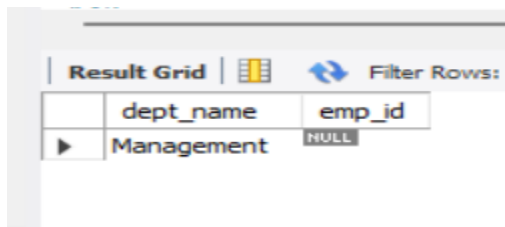


The screenshot shows a 'Result Grid' window with a table containing 7 rows of project assignment data. The table has columns for project_name, emp_name, and role. The data is as follows:

project_name	emp_name	role
Payroll System	Amit Sharma	Developer
Payroll System	Priya Singh	HR Support
E-Commerce Portal	Amit Sharma	Developer
E-Commerce Portal	Rahul Verma	Manager
E-Commerce Portal	Neha Joshi	Tester
Mobile App	Sonia Mehta	Developer

4. Departments with no employees (RIGHT JOIN)

```
select departments.dept_name, employees.emp_id from employees  
right join departments on employees.dept_id = departments.dept_id  
where employees.emp_id is null;
```



	dept_name	emp_id
▶	Management	NULL

5. Full join alternative using UNION

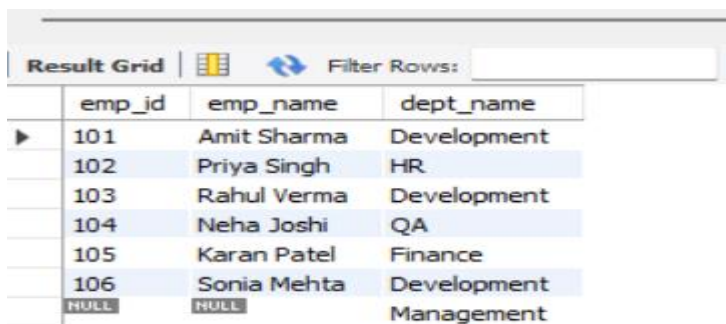
```
select employees.emp_id, employees.emp_name, departments.dept_name  
from employees
```

```
left join departments on employees.dept_id = departments.dept_id
```

```
union
```

```
select employees.emp_id, employees.emp_name, departments.dept_name  
from employees
```

```
right join departments on employees.dept_id = departments.dept_id;
```



	emp_id	emp_name	dept_name
▶	101	Amit Sharma	Development
	102	Priya Singh	HR
	103	Rahul Verma	Development
	104	Neha Joshi	QA
	105	Karan Patel	Finance
	106	Sonia Mehta	Development
	NULL	NULL	Management

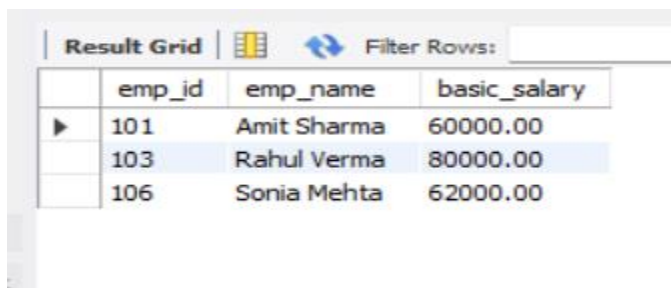
F. Subqueries

1. Employees earning more than company avg

```
select employees.emp_id, employees.emp_name, salaries.basic_salary from  
employees
```

```
join salaries on employees.emp_id = salaries.emp_id
```

```
where salaries.basic_salary > (select avg(basic_salary) from salaries);
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains three rows of data with columns 'emp_id', 'emp_name', and 'basic_salary'. The second row, representing Rahul Verma with a salary of 80000.00, is highlighted in blue.

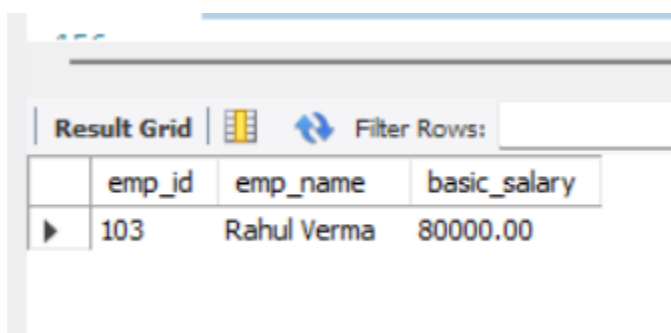
	emp_id	emp_name	basic_salary
▶	101	Amit Sharma	60000.00
	103	Rahul Verma	80000.00
	106	Sonia Mehta	62000.00

2. Highest earning employee

```
select employees.emp_id, employees.emp_name, salaries.basic_salary from  
employees
```

```
join salaries on employees.emp_id = salaries.emp_id
```

```
where salaries.basic_salary = (select max(basic_salary) from salaries);
```

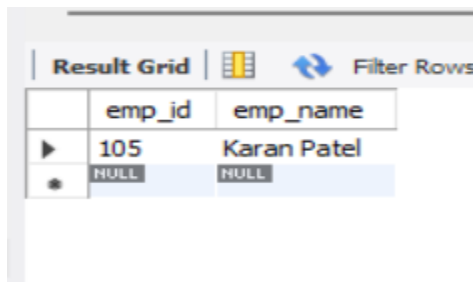


The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains a single row of data with columns 'emp_id', 'emp_name', and 'basic_salary'. The row, representing Rahul Verma with a salary of 80000.00, is highlighted in blue.

	emp_id	emp_name	basic_salary
▶	103	Rahul Verma	80000.00

3. Employees not assigned to any project

```
select emp_id, emp_name from employee  
where emp_id not in (select emp_id from employee_projects);
```

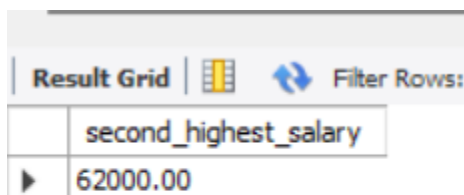


	emp_id	emp_name
▶	105	Karan Patel
●	NULL	NULL

G. Nested Subqueries

1. Second highest salary

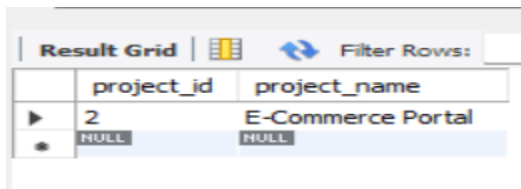
```
select max(basic_salary) as second_highest_salary from salaries  
where basic_salary < (select max(basic_salary) from salaries);
```



	second_highest_salary
▶	62000.00

2. Projects with more than 2 employees

```
select project_id, project_name from projects  
where project_id in (select project_id from employee_projects  
group by project_id having count(emp_id) > 2);
```

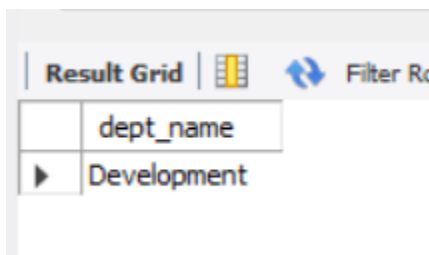


The screenshot shows a SQL query result grid with two columns: project_id and project_name. The first row contains the values 2 and E-Commerce Portal. The second row contains NULL and NULL.

project_id	project_name
2	E-Commerce Portal
NULL	NULL

3. Departments having avg salary > finance

```
select departments.dept_name from departments  
join employees on departments.dept_id = employees.dept_id  
join salaries on employees.emp_id = salaries.emp_id group by  
departments.dept_name  
having avg(salaries.basic_salary) > (select avg(s.basic_salary) from  
employees e  
join salaries s on e.emp_id = s.emp_id  
join departments d on e.dept_id = d.dept_id where d.dept_name =  
"Finance");
```



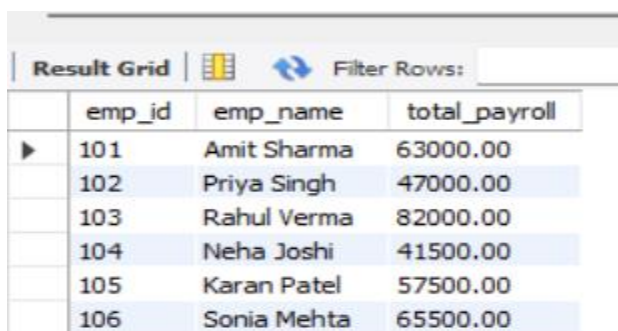
The screenshot shows a SQL query result grid with one column: dept_name. The first row contains the value Development.

dept_name
Development

H. Advanced (Optional)

1. Total payroll cost per employee

```
select employees.emp_id, employees.emp_name,  
  
(salaries.basic_salary + salaries.bonus - salaries.deductions) as total_payroll  
  
from employees join salaries on employees.emp_id = salaries.emp_id;
```

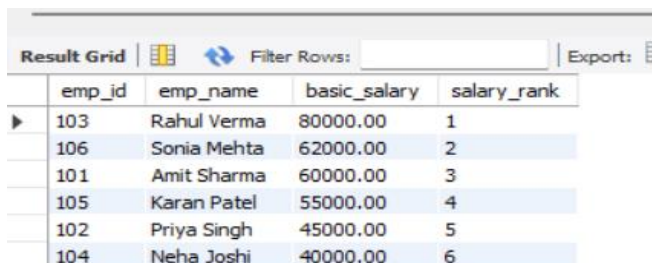


The screenshot shows a 'Result Grid' with a toolbar containing a 'Filter Rows' button and a search input field. The grid displays the following data:

	emp_id	emp_name	total_payroll
▶	101	Amit Sharma	63000.00
	102	Priya Singh	47000.00
	103	Rahul Verma	82000.00
	104	Neha Joshi	41500.00
	105	Karan Patel	57500.00
	106	Sonia Mehta	65500.00

2. Rank employees by salary (window)

```
select employees.emp_id, employees.emp_name, salaries.basic_salary,  
  
rank() over (order by salaries.basic_salary desc) as salary_rank from  
employees  
  
join salaries on employees.emp_id = salaries.emp_id;
```

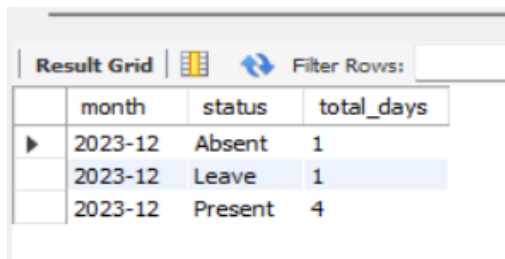


The screenshot shows a 'Result Grid' with a toolbar containing a 'Filter Rows' button, a search input field, and an 'Export' button. The grid displays the following data:

	emp_id	emp_name	basic_salary	salary_rank
▶	103	Rahul Verma	80000.00	1
	106	Sonia Mehta	62000.00	2
	101	Amit Sharma	60000.00	3
	105	Karan Patel	55000.00	4
	102	Priya Singh	45000.00	5
	104	Neha Joshi	40000.00	6

3. Month-wise attendance summary

```
select date_format(date, '%Y-%m') as month, status,  
count(*) as total_days from attendance  
group by date_format(date, '%Y-%m'), status order by month;
```



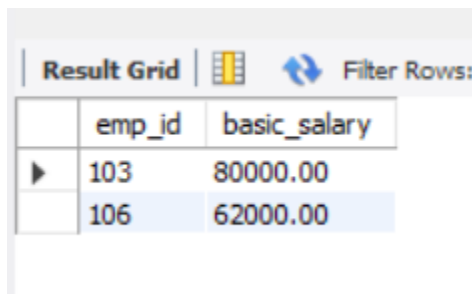
The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with three columns: 'month', 'status', and 'total_days'. The data is grouped by month and status. The first row shows '2023-12' with 'Absent' status and a count of 1. The second row shows '2023-12' with 'Leave' status and a count of 1. The third row shows '2023-12' with 'Present' status and a count of 4.

	month	status	total_days
▶	2023-12	Absent	1
	2023-12	Leave	1
	2023-12	Present	4

I. Views

1. High earners

```
create view high_earners as select emp_id, basic_salary from salaries  
where basic_salary > 60000;  
select * from high_earners;
```



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with two columns: 'emp_id' and 'basic_salary'. The data shows two employees: one with emp_id 103 and basic_salary 80000.00, and another with emp_id 106 and basic_salary 62000.00.

	emp_id	basic_salary
▶	103	80000.00
	106	62000.00

2. Department-wise salary summary

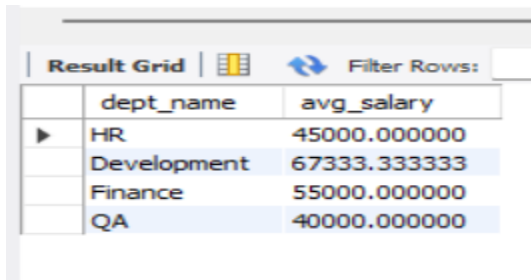
```
create view dept_salary_summary as select d.dept_name, avg(s.basic_salary)
as avg_salary from employees e
```

```
join salaries s on e.emp_id = s.emp_id
```

```
join departments d on d.dept_id = e.dept_id
```

```
group by d.dept_name;
```

```
select * from dept_salary_summary;
```



dept_name	avg_salary
HR	45000.000000
Development	67333.333333
Finance	55000.000000
QA	40000.000000

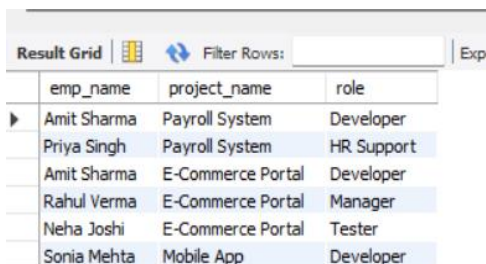
3. Employee project allocation

```
create view employee_project_view as select e.emp_name, p.project_name,
ep.role from employees e
```

```
join employee_projects ep on e.emp_id = ep.emp_id
```

```
join projects p on p.project_id = ep.project_id;
```

```
select * from employee_project_view;
```



emp_name	project_name	role
Amit Sharma	Payroll System	Developer
Priya Singh	Payroll System	HR Support
Amit Sharma	E-Commerce Portal	Developer
Rahul Verma	E-Commerce Portal	Manager
Neha Joshi	E-Commerce Portal	Tester
Sonia Mehta	Mobile App	Developer

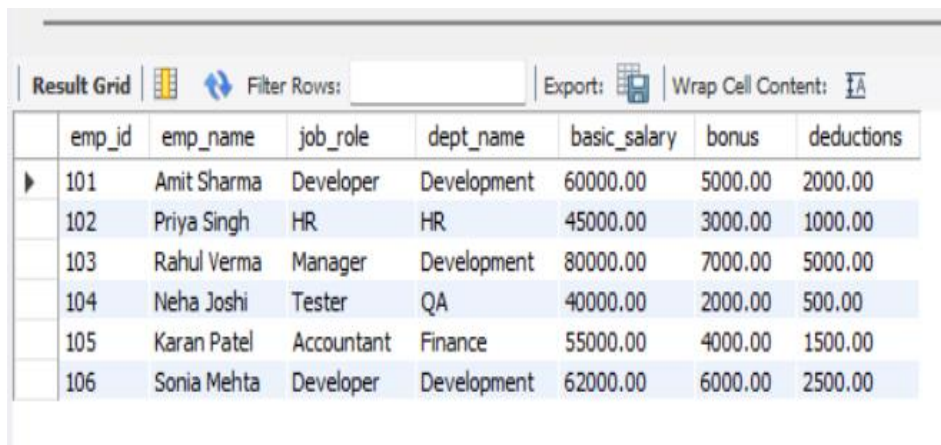
4. Employee Full Profile View

create view employee_full_profile as select e.emp_id, e.emp_name, e.job_role, d.dept_name, s.basic_salary, s.bonus, s.deductions from employees e

join departments d on e.dept_id = d.dept_id

join salaries s on e.emp_id = s.emp_id;

select * from employee_full_profile;



The screenshot shows a database query result grid with a toolbar at the top containing 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The grid displays a table with 8 columns: emp_id, emp_name, job_role, dept_name, basic_salary, bonus, and deductions. There are 6 rows of data, each representing an employee's full profile.

	emp_id	emp_name	job_role	dept_name	basic_salary	bonus	deductions
▶	101	Amit Sharma	Developer	Development	60000.00	5000.00	2000.00
	102	Priya Singh	HR	HR	45000.00	3000.00	1000.00
	103	Rahul Verma	Manager	Development	80000.00	7000.00	5000.00
	104	Neha Joshi	Tester	QA	40000.00	2000.00	500.00
	105	Karan Patel	Accountant	Finance	55000.00	4000.00	1500.00
	106	Sonia Mehta	Developer	Development	62000.00	6000.00	2500.00

5. Employees Without Project View

create view employees_without_projects as select e.emp_id, e.emp_name from employees e

left join employee_projects ep on e.emp_id = ep.emp_id

where ep.project_id is null;

select * from employees_without_projects;

Result Grid			Filter Rows:
	emp_id	emp_name	
▶	105	Karan Patel	

6. Monthly Attendance Summary View

```
create view monthly_attendance_summary as select emp_id,
date_format(date, '%Y-%m') as month,
status, count(*) as total_days from attendance
group by emp_id, date_format(date, '%Y-%m'), status;
select * from monthly_attendance_summary;
```

Result Grid					Filter Rows:
	emp_id	month	status	total_days	
▶	101	2023-12	Present	1	
	102	2023-12	Absent	1	
	103	2023-12	Present	1	
	104	2023-12	Leave	1	
	105	2023-12	Present	1	
	106	2023-12	Present	1	

J. DML Operations

1. Update salary

set SQL_SAFE_UPDATES=0;

update salaries set basic_salary = 65000 where emp_id = 101;

select * from salaries;

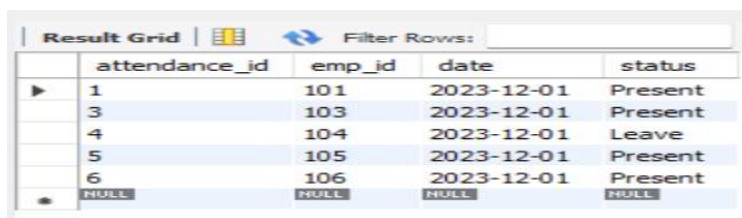


	salary_id	emp_id	basic_salary	bonus	deductions
▶	1	101	65000.00	5000.00	2000.00
	2	102	45000.00	3000.00	1000.00
	3	103	80000.00	7000.00	5000.00
	4	104	40000.00	2000.00	500.00
	5	105	55000.00	4000.00	1500.00
	6	106	62000.00	6000.00	2500.00
●	NULL	NULL	NULL	NULL	NULL

2. Delete all “Absent” attendance

delete from attendance where status = "Absent";

select * from attendance;



	attendance_id	emp_id	date	status
▶	1	101	2023-12-01	Present
	3	103	2023-12-01	Present
	4	104	2023-12-01	Leave
	5	105	2023-12-01	Present
	6	106	2023-12-01	Present
●	NULL	NULL	NULL	NULL

3. Insert a new project

```
insert into projects (project_id, project_name, start_date, end_date)
```

```
values (4, "AI Chatbot System", "2024-01-01", "2024-08-31");
```

```
select * from projects;
```

Result Grid				
Filter Rows:				
Edit:				
	project_id	project_name	start_date	end_date
▶	1	Payroll System	2023-01-01	2023-06-30
	2	E-Commerce Portal	2023-02-01	2023-09-30
	3	Mobile App	2023-03-15	2023-12-31
	4	AI Chatbot System	2024-01-01	2024-08-31
•	NULL	NULL	NULL	NULL

4. Transfer an employee to another department

update employees set dept_id = 4 where emp_id = 106;

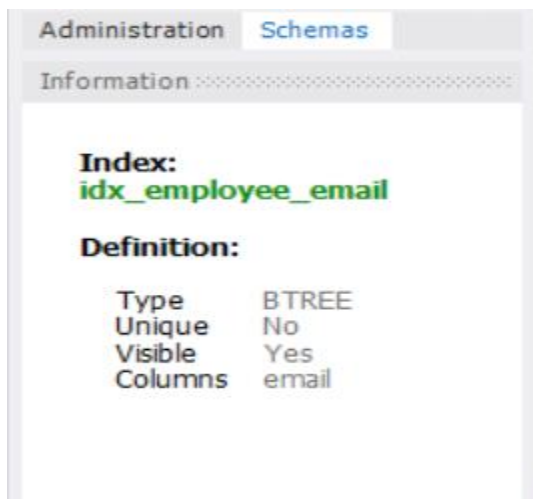
```
select * from employees;
```

Result Grid		 Filter Rows:		Edit:				Export/Import:			View
	emp_id	emp_name	email	phone	hire_date	dept_id	job_role				
▶	101	Amit Sharma	amit@company.com	9876543210	2022-01-10	2	Developer				
	102	Priya Singh	priya@company.com	9922334455	2021-03-15	1	HR				
	103	Rahul Verma	rahul@company.com	8811223344	2020-07-22	2	Manager				
	104	Neha Joshi	neha@company.com	9988776655	2023-02-10	4	Tester				
	105	Karan Patel	karan@company.com	9090909090	2019-11-01	3	Accountant				
	106	Sonia Mehta	sonia@company.com	8899001122	2021-04-19	4	Developer				
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL				

K. Constraints & Index

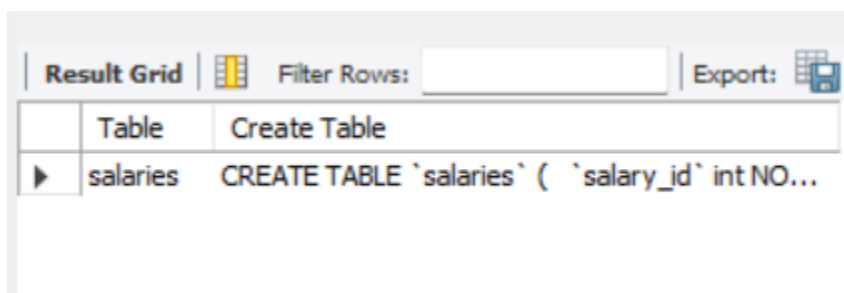
1. Create index on email

```
create index idx_employee_email on employees(email);
```



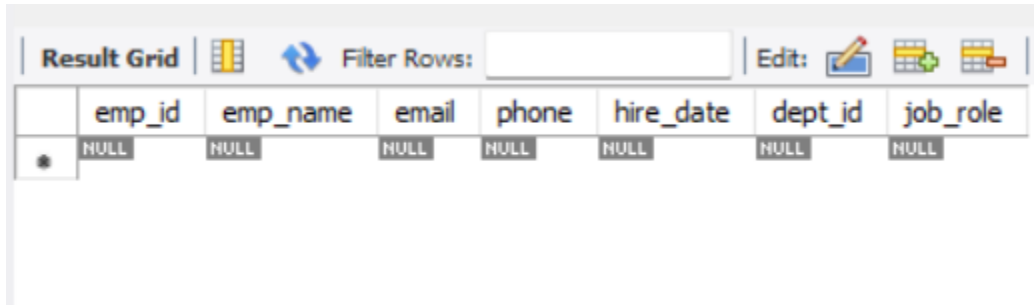
2. Add CHECK constraint for min salary (age $\geq$ 18)

```
alter table salaries add constraint chk_min_salary check (basic_salary >= 18000);
```



3. Add NOT NULL to phone column

select * from employees where phone is null;



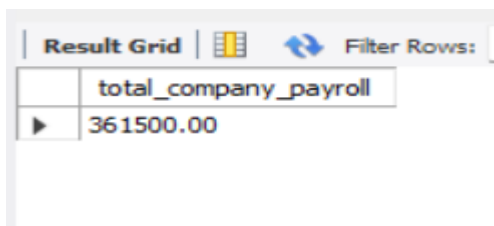
The screenshot shows a 'Result Grid' window with a toolbar at the top containing icons for 'Filter Rows', 'Edit', and other grid functions. Below the toolbar is a table with 8 columns: emp_id, emp_name, email, phone, hire_date, dept_id, and job_role. The first row of data shows all values as NULL, indicating that no employees were found with a null phone number.

	emp_id	emp_name	email	phone	hire_date	dept_id	job_role
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

L. Case Study Queries - Internet reference

1. Total payroll cost of company

select sum(basic_salary + bonus - deductions) as total_company_payroll
from salaries;



The screenshot shows a 'Result Grid' window with a toolbar at the top. Below the toolbar is a table with one column: total_company_payroll. The first row of data shows the value 361500.00.

	total_company_payroll
▶	361500.00

2. Employee working on max projects

select e.emp_id, e.emp_name, count(ep.project_id) as project_count from
employees e

join employee_projects ep on e.emp_id = ep.emp_id

group by e.emp_id, e.emp_name

```

having count(ep.project_id) = (select max(project_count)
from (select count(project_id) as project_count
from employee_projects group by emp_id) as temp);

```

Result Grid			
Filter Rows:			
	emp_id	emp_name	project_count
▶	101	Amit Sharma	2

3. Department with highest employee count

```

select d.dept_name, count(e.emp_id) as emp_count from departments d
join employees e on d.dept_id = e.dept_id
group by d.dept_name order by emp_count desc limit 1;

```

Result Grid		
Filter Rows:		
	dept_name	emp_count
▶	Development	2

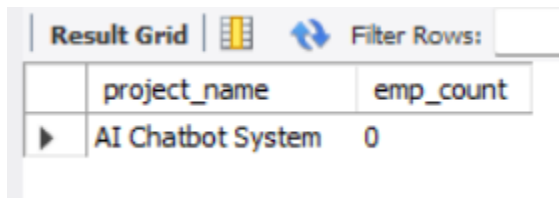
4. Least busy project

```

select p.project_name, count(ep.emp_id) as emp_count from projects p
left join employee_projects ep
on p.project_id = ep.project_id
group by p.project_id, p.project_name

```

order by emp_count asc limit 1;



The screenshot shows a 'Result Grid' interface. At the top, there is a 'Filter Rows:' input field. Below it, a table displays the results. The table has two columns: 'project_name' and 'emp_count'. The first row shows 'AI Chatbot System' in the 'project_name' column and '0' in the 'emp_count' column.

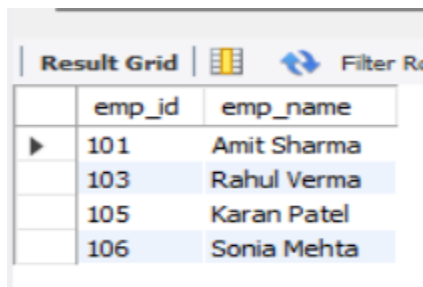
project_name	emp_count
AI Chatbot System	0

5. Employees with 100% attendance

select e.emp_id, e.emp_name from employees e

join attendance a on e.emp_id = a.emp_id

group by e.emp_id, e.emp_name having sum(case when a.status <>
"Present" then 1 else 0 end) = 0;



The screenshot shows a 'Result Grid' interface. At the top, there is a 'Filter Rows:' input field. Below it, a table displays the results. The table has two columns: 'emp_id' and 'emp_name'. The first four rows show employees with 100% attendance: 101 (Amit Sharma), 103 (Rahul Verma), 105 (Karan Patel), and 106 (Sonia Mehta).

emp_id	emp_name
101	Amit Sharma
103	Rahul Verma
105	Karan Patel
106	Sonia Mehta

Summary of learnings

1. I learned how to design a proper relational database using multiple tables with primary keys and foreign key relationships.
2. I practiced writing SQL queries from basic to advanced level using SELECT, WHERE, ORDER BY, and aggregate functions.
3. I understood how GROUP BY and HAVING are used for department-wise and salary-wise data analysis.
4. I learned different types of joins and their use cases such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN using UNION.
5. Subqueries and nested queries helped me solve real-world problems like finding the highest salary and project allocation.
6. I created reusable SQL views to simplify complex queries and improve code readability.
7. I performed DML operations including INSERT, UPDATE, DELETE, and employee department transfers safely.
8. I learned how constraints and indexes help maintain data integrity and improve query performance.
9. Advanced SQL queries such as payroll calculation, employee ranking, and attendance analysis gave me practical hands-on experience.